

Authentication and Authorization

Authentication and Authorization

Authentication and authorization. These are the most important parts of your backend system and it's parts that developers genuinely understand the least. In all reality, the code for your app is such a minimal part of your backend, but I would say that it has the biggest impact. If you have a bug in your business logic, that might just break a feature, but a bug in your authentication breaks trust, privacy, and legal compliance, which is big No, no. And when you're doing like some sort of happy path testing, you're not going to necessarily see any off bugs. And even though you're not seeing these bugs, attackers of your system are specifically only looking for these kinds of bugs. So, it's imperative that we break this down to a level that everyone can understand and implement into their own systems.

If you think Auth is done when login works, then that's where real problems are going to start. A login on your app is just a single event. Authorization should happen on every request and systems begin to fail when Auth sort of gets treated like a checkbox so to speak.

Authentication answers who you are once. Authorization decides what you can do thousands of times.

This is a key attribute right here. Authentication answers who you are once. But authorization is going to decide what you can do thousands of times across every request.

Why Authentication and Authorization Bugs Are So Serious

*Feature bugs = annoying
Auth bugs = data leaks, lawsuits, loss of users*

And Auth bugs are absolutely catastrophic. Feature bugs. These are just annoying things that you have to deal with. But if you have Auth bugs, you're going to deal with data leaks, lots of users, and even lawsuits if you're working in some sort of field that has compliance regulations, etc. Most major breaches are authorization failures, not cryptography failures.

Why Developers Underestimate Auth

So, why does this happen? Why do devs underestimate Auth?

Frameworks make auth look easy

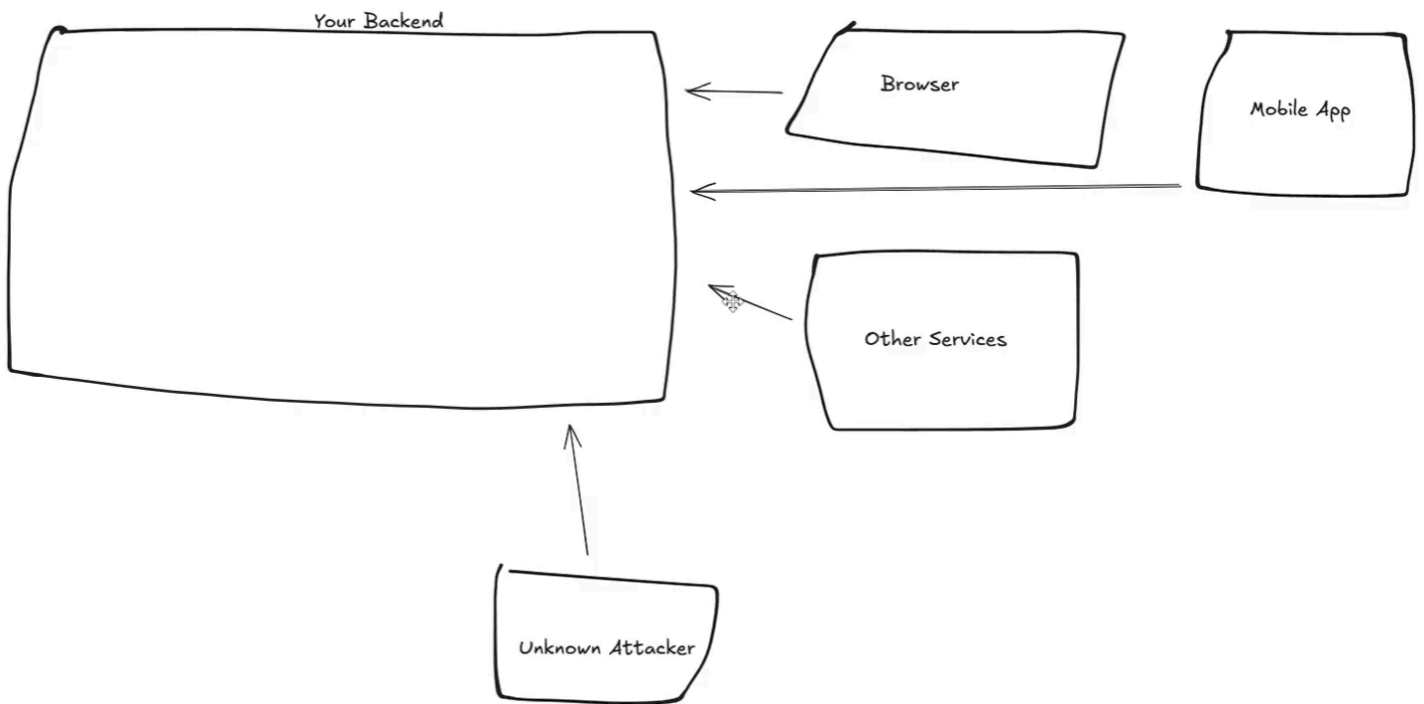
Tutorials stop at login

JWTs create false confidence

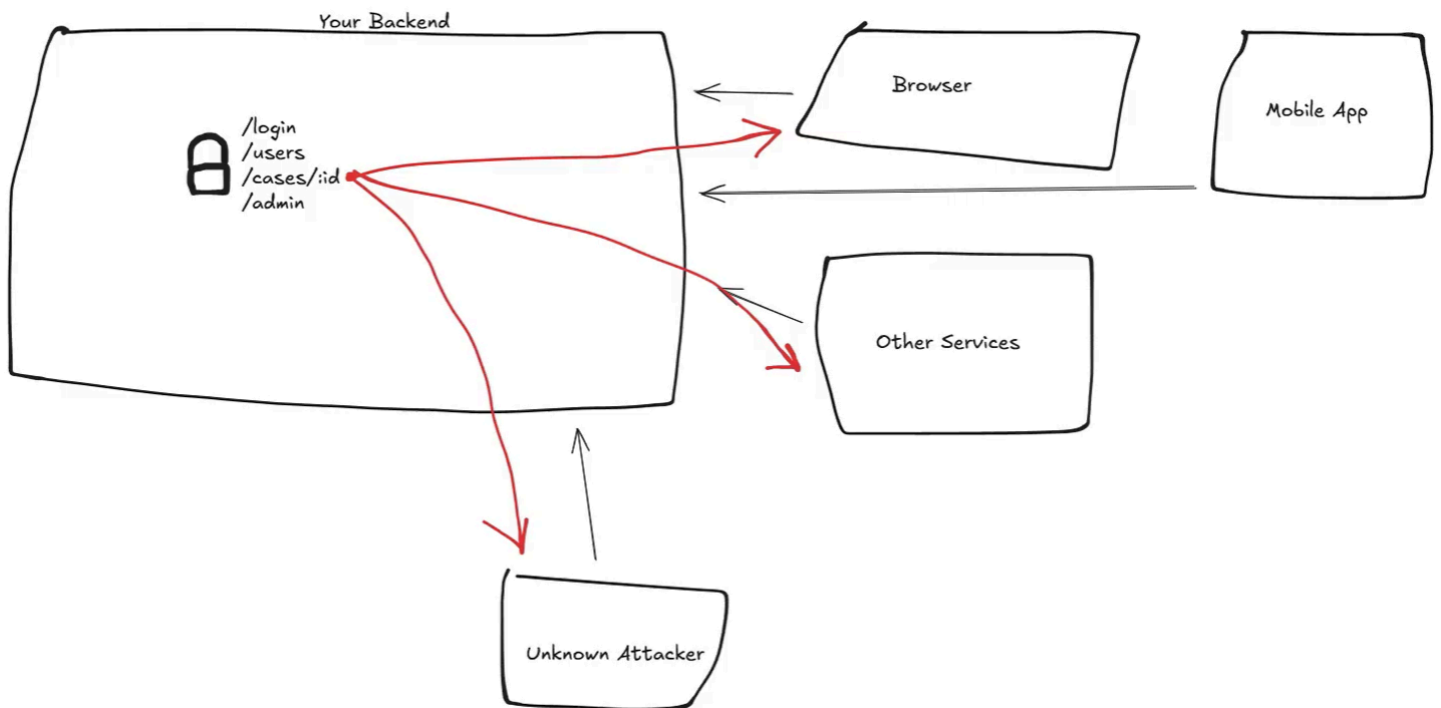
Happy-path thinking ("why would someone do that?")

Well, there's probably a lot of reasons, but I would say the main ones are going to be the fact that frameworks make it look easy and they abstract a lot of it away. Um, a lot of tutorials, at least that I've seen, stop at just logging in. And JWTs, which we'll get into, create this sort of sense of false confidence that I'll talk about later. And you can't have this like happy path ignorant way of thinking like, why would someone do that? Why would an attacker try to compromise my system? You know, you have to account for all edge cases when it comes to Auth and attackers, they're not going to use your UI. Instead, they're going to attack the API.

Authentication and Authorization as a Cross-Cutting Concern



So, at a very high level, this is a representation of your backend right here. And all of these arrows into your backend, these can represent requests of some sort. These are representations of a potential attack surface.



Anytime someone is sending a request to make your backend do something, it's a potential to do something malicious. And this isn't just a feature. Authorization authentication is not a feature. It's a crosscutting concern. Lots of routes, they should be locked up very, very tightly. I'm artistically challenged. So just assume that this red dot represents a bug. And this bug can lead to something like this (red lines). and you know, you start to leak other people's data, sensitive data, and it's just it's very messy. And really, one missing authorization check can expose everything. So, it's not hard because of crypto. It's not hard because we need to encrypt the messages. It's hard because humans build complex systems and attackers only need one mistake to capitalize off of to completely compromise your system.

Authentication vs. Authorization

Authentication answers who you are.
Authorization answers what you're allowed to do.

But before we go any further, we need to lock in the most important distinction of this entire video. Right? So to start off, if we can cleanly define the difference between authentication and authorization is going to make the rest of this video much easier. So authentication essentially answers who you are and authorization answers what you're allowed to do. What kind of permissions do you have in this back end? They're related, but they solve entirely different problems.

Why the Distinction Matters

Logged in != admin

Logged in != owner of a resource

Logged in != same permissions in every organization

So why does this distinction matter? Why can't we just bunch this into one thing? Well, I want you to take a look at some of these. Like obviously if a user is logged in, that doesn't necessarily make them an administrator. It doesn't necessarily make them an owner of a specific resource. And it doesn't mean that they have the same permissions across every different organization in your app.

And so you can be authenticated. The server can know who you are, but that doesn't mean that you're authorized. And success does not automatically mean that you should be granted access to specific resources. And most real world problems come after authentication. when this authorization layer is neglected. Authentication is a binary thing. You either are authenticated or you're not. But authorization is contextual. There can be several roles, several different permissions across your entire app. And we need to ensure on each request that whoever this authenticated user is has those permissions. So with that said, obviously authentication is like an ID.

Authentication as Identity

Authentication is like an ID. It proves your identity, but the doors really open on authorization when we check to see if you have access to a resource depending on your location, the specific time when you try to access that resource, and what resource you're even trying to get access to. Authorization depends on a sort of context where authentication really doesn't. Some that I see come up time and time again is treating JWT validation as authorization, not separating the two, checking authorization just once at login, assuming roles are universal, or even putting permission logic in the front end. This is this is a big no no. Please don't do this, guys. And we'll get into role-based access control and permission logic later in the video, but I want you guys to remember like if your authorization logic lives in one place, it's probably wrong. So here's like the simplest clear distinction between the two.

The Simple Distinction

Authentication is going to answer a question like:-

Who are you? This deals with login, tokens, and sessions.

Whereas authorization answers the question:-

What can you do? This is done on every request. It's resource-based and it's contextual. Making sure that whoever this user is has access to the resource that they're trying to get access to.

Request → Authentication → Authorization → Resource
Identity check Permission check

So this is a very simple request flow. User sends a request, they go through the authentication process, authorization process, and then they get access to the resource. Authentication is an identity check and authorization is a permission check. And both of these must pass every time.

Request → Authentication → ~~Authorization~~ → Resource
Identity check Permission check

Suppose we miss this layer, but we have a bug and these requests go through anyway. If you're missing authorization, this will lead to a data leak. And most systems don't fail because Auth doesn't exist. They fail because authorization is incomplete often times.

But this is the core message at a basic level.

Authentication gets you in the building.

Authorization decides which rooms you're allowed to enter.

Subjects, Identities, Accounts, and Context

But to go deeper, we really need a better language because user is honestly a very vague concept. By the end of this section, I'm hoping that you as the viewer can stop thinking in only just terms of users, but understand **what a subject actually is** and see **why context changes authorization decisions**.

What Is a Subject?

So most developers, they say the word user, but they actually say this word for three different things.

User != account != identity

Often they like envelop the word user for all three of these things, but the user does not equal the account. The account does not equal the identity.

Auth systems don't care about humans, they care about subjects

Your Auth system doesn't care about humans. They don't care about who actually is the person on the other end of the computer actually is. They care about subjects.

Your system, it's not authenticating people necessarily. It's authenticating identities. It's like what is a subject, right? A subject is anything that can make a request. Some examples of subjects are **human users, back-end**

services, cron jobs, third party integrations, and mobile apps. And here's the important lesson. **Every** subject needs authentication, and not every subject is a person. Super important.

Identity vs. Account

Identity -> proof of who/what you are

Account -> representation of your system

So the next most important distinction in this section is identity versus account. Identity is simply a proof of who or what you are. Account on the other hand is a representation of your system.

One identity → multiple accounts

One account → multiple identities (SSO, OAuth)

One identity can represent multiple accounts. One account can be multiple identities. But none of this matters without context.

Authorization Depends on Context

And honestly, context changes everything. Something I want you guys to understand about context. Authorization is not global.

Authorization is NOT global

Permissions depend on context:

Organization

Tenancy

Resource

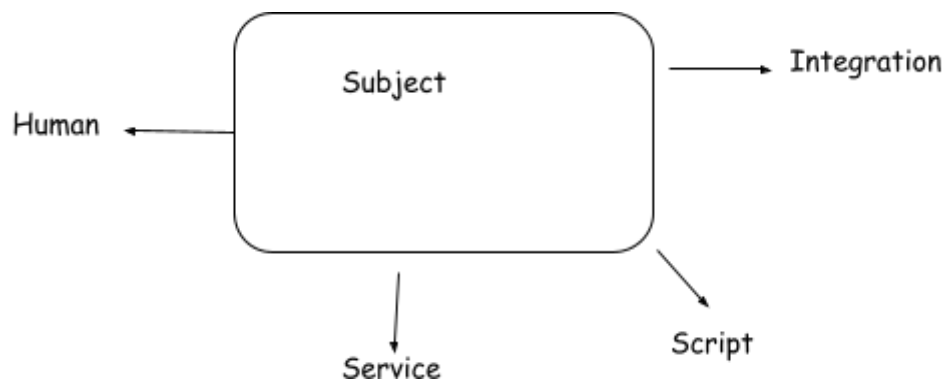
Action

The same identity can be an admin in one context, but a viewer in another

And permissions, they depend on context. things like what organization is this user a part of, tenancy, what resource are they trying to access, what specific action are they trying to perform. The same identity can be an admin in one context but a viewer in another. Think about it, right? Imagine you have some sort of backend application with different organizations and you have a user that's a part of two organizations and in one organization they are an admin and in another they are just a base user base viewer. Obviously we need a clean separation of permissions depending on what resources they're trying to access based on what organization those resources belong to. And so authorization ties into that and that's the main reason it's not global.

The Subject and Context Model

So let me draw a box for this. And this box is going to be our subject.



And this subject can like branch off and be multiple different things. It's going to be a human being, service script, can even be an integration. All systems they don't care which one of these it is. They just need to figure out what it is.

proven stored evaluated
 Identity → Account → Permissions

The identity needs to be proven. And then we get to the account which is stored. And then finally we evaluate the permissions of that proven identity that is a part of this stored account whichever account they're on. And things like specific organizations, resources or actions.

Identity
 ↓
 [Context] (Org, Resource, Action)
 ↓
 Authorization Decision

These all live inside this context bubble. So hopefully now you understand why authorization without context is meaningless because we have an identity. We need to figure out the context of whom this identity is and then finally make a final authorization decision based on that. So yeah, authors, it's about subjects within a context.

Authentication Factors

So the next question is how do they prove it? Now we get to talk about my favorite section when it comes to this topic which is authentication factors. So authentication obviously is the process of proving an identity because any machine can claim to be whoever or whatever they want but we need to prove that in some way and every authentication system is just an evaluation device to figure out are you actually what you say you are.

The Three Authentication Factors

Something you know

Passwords

PINs

Security questions (bad)

Something you have

Phone

Hardware token

Authenticator app

Session cookie

Something you are

Biometrics

Fingerprint

Face ID

It can be **something that you know** like a password or a pin or a security question. It can be **something that you have or physically own** like your phone, a hardware token, or an authentication app on your phone or a session cookie or it can be **something that you are**. Biometrics, fingerprint scans, and Face ID are all examples of this. And the more factors that you mix and match and combine that are required for authentication, the harder it becomes to impersonate someone or some machine.

Why Single-Factor Authentication Fails

Password reuse

Phishing

Credential stuffing

Database leaks

So now we can talk about why single factor auth fails. These can all be examples of failure points. So obviously if you reuse the same password across all of your accounts and your password is in a data breach database leak, this ties into it. Obviously, attackers, if they find out that you have an account on one platform, maybe YouTube, and they see that your password on YouTube is I love bears 123, naturally, they're going to start going to your bank accounts and trying that password to see if it is the same password on your other accounts.

Phishing Attacks

Fishing attacks. Someone might send you a malicious email that looks like it's from your bank, but it's really fraudulent email from the attacker in order to get you to send them their credentials.

Credential Stuffing

And then we have credential stuffing, which attackers just get a large sum of information of authentication information and then brute force and try to log in that way. Most attackers, they're not cracking passwords. They're not cracking cryptography or breaking passwords. They're just reusing them. And you may think that you're safe, but your password that you think is so secure can even fail at scale. And attackers automate everything so that they can work extremely quickly and extremely efficiently.

Multi-Factor Authentication

So this is why multifactor authentication is so important. This is one of the strongest risk reduction mechanisms. It adds more friction, sure, but the security benefits are way better and way more important in my opinion. you really have some leniency and some play with how you set this up,

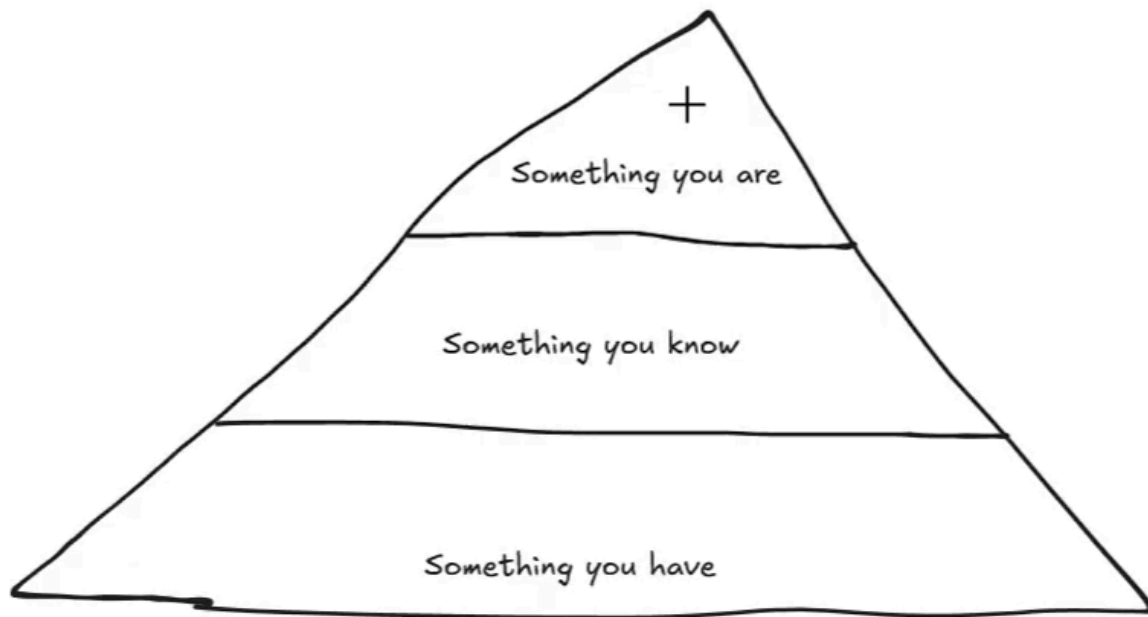
Admin users → mandatory MFA

Regular users → optional MFA

Sensitive actions → step-up auth

but I think that admin users should have mandatory multifactor authentication. Regular users can be optional and sensitive actions can require some sort of step up auth for sure and they should be honest. Your authentication strength should match the risk of the action that's being taken.

Authentication Factor Pyramid



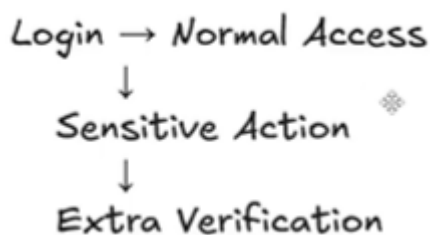
So let me represent this by a factor pyramid. So security here increases as we go up. You know something you have someone can steal your phone. Someone can steal if they steal your phone they'll have access to your authentication app. This is the most insecure factor bucket.

The step up from that is something you know passwords pins etc. And then obviously the step up from that is something that you are certain biometric scans your fingerprint your face ID. These are the most secure factors.

Step-Up Authentication

Attacker → password leak → MFA BLOCK → account access

So this is a very simple example of an attack path. An attacker gains access to a password in some sort of leak and then they get access to your account. But if we have some sort of multifactor authentication in our system that will block access even if they have access to this password which is in the middle of this factor pyramid. Maybe it's a 2FA passcode to your email, maybe it's Face ID, etc. If we have this block, it raises the cost for this attacker to gain account access. So this is a very basic example of how most setup off is going to work, right? User logs in, they authenticate themselves and they get normal access.



But if they want to perform some sort of sensitive action, it requires extra verification. An example of this, if you've ever used GitHub, if you want to go and change the visibility of a repository from either public to private or vice versa, you have to go through some sort of 2FA to do that. But not every action needs the same level of trust, which is why it's set up this way.

Passwords

But let's now talk about perhaps the weakest and most misunderstood factor, which is passwords. So there's a big myth with passwords and I want to remove this idea from your brains right now. I want you to understand something very quickly. If you store passwords encrypted, your system is already broken.

Why Passwords Should Not Be Encrypted

Why is that? You might hear that and you might think, well, if it's encrypted, then isn't it safe? No, it's not safe. Reason for that being the fact that encryption is reversible. Password storage must be a one-way transaction. If you can decrypt the password, then guess what? The attacker can also decrypt the password. If your system has the capability to recover a password from an encrypted state, then an attacker can do the same. And it's unsafe by design.

Hashing vs. Encryption



So this is where we need to talk about a key difference. The difference between hashing and encryption. So what are the distinctions? Well, encryption is reversible. It requires some sort of key and it's used for secrets that need to be recoverable. Whereas hashing, it's a one-way transaction. It's a one-way street without any sort of key. And it's used for verification. Passwords should never be recovered. They should only be verified.

Example of Encryption

Encrypt Online

Encrypt text, strings, JSON, YAML, config files and more!

Encryption Type

aes-256-cbc

Your input was encrypted using aes-256-cbc and the passphrase you supplied, then base64 encoded. The result is below.

Copy

this is some super sensitive data

U2FsdGVkX1/+OMWilmdRHxNRdqS3il7WwyE6UkFc2mKTO8ICTOEQ5w2cWC1S+K0wudJbhADN2//2pgNdUHluA==

ilovedogs123

Generate passphrase

Use the [decrypt tool](#) to decrypt this value, or use [OpenSSL 1.1.0](#) on your local computer.

Encrypt

Decryption Type

aes-256-cbc

Your input was decrypted using aes-256-cbc and the passphrase you supplied.
The result is below.

Copy

U2FsdGVkX1/+OMWilmDRHxNRdqS3il7WwyE6UkFc2mKTO8ICTOEQ5w2cWC1S+
+K0wudJbhADN2//2pgNdUHLuA==

this is some super sensitive data

Ilovedogs123

I

To learn more about decryption using OpenSSL, see the [OpenSSL manpages](#). Use the [encrypt tool](#) to encrypt another value.

Decrypt

Here's an example of encryption. Okay? And this is why this is why it's negative for password storage. Let's go through a basic example of encryption real quick. If I say this is some super sensitive data and we need a passphrase to encrypt this. So if my passphrase is I love dogs 1 2 3 and I encrypt this data. This is now the encrypted version of this text. Looks pretty random, right? What are you going to do with this?

Well, if you have this passphrase, you can decrypt this data. So if I enter this here and I say I love dogs 123, what do you know? This is some super sensitive data and now we have access to it.

Example of Hashing

☰

SHA256 ▾

Calculate SHA-256 hashes or HMAC-SHA-256 for text using UTF-8, UTF-16, Hex, Base64, and other input encodings. [Read more](#) ▾

Input ↕

this is my super sensitive data

Hash

Advertisement ▾

Output ↕

c3c407b06740ff2efa47c2e77ea4c23064b0974b63993cef862d2453451c0545

Whereas if we hash it and we use some sort of hashing algorithm and I'm like "this is my super sensitive data" is going to hash this and this is fundamentally irreversible because it's algorithm based and it's not passkeybased.

Salting Passwords

But if you're adept, you might have noticed a problem with hashing as well. And there's a reason that makes plain hashing not enough. Fast hashes like secure hashing algorithm 256, especially in the modern era. We have GPUs that can try billions of hashes per second. There are also tools like rainbow tables that I won't get too deep into. How do we how do we make hashing more secure and make it secure enough for production grade apps where security is top priority? That's where salting comes in.

What Is Salting?

So whenever you hash some sort of password, it needs to go through some sort of salting round. And salting is just adding some sort of random unique string to a password to the password string before it's hashed. And this creates a distinct hash for users with the same password. So let's say we have two users and both of their passwords are I love dogs 1 2 3.

If we just use standard hashing then the hash for their passwords is going to be the same. But with salting when we add a random unique string to each person's password the hashes become different.

Bcrypt and Salt Rounds

Generate Hash

Generate a bcrypt hash from your text. Higher rounds provide better security but take longer to process.

Text to Hash

I love dogs 123

Rounds (Cost Factor): 12

-

+

High security - suitable for production

Generate Hash

\$2a\$12\$aBMKx9Nk9RuEPbBETpQyW.eB8hMKJPTbGiR7jrTxw6eFU9Rx2ITkq

Copy Hash

Generated with 12 rounds of hashing

So this is an online brypt hash generator. B-crypt very common tool that people use when they're trying to hash passwords in their apps. As you can see, I've hashed the password I love dogs 123 at 12 salt rounds. So, the number of rounds is essentially just the amount of times that this text undergoes the hashing algorithm.

And it's not actually 12. Things like bcrypt undergo two to the power of the number of rounds. So, in this example, we're actually salting the password two to the power of 12 times. And if you notice, look at this string.

Generate Hash

Generate a bcrypt hash from your text. Higher rounds provide better security but take longer to process.

Text to Hash

I love dogs 123

Rounds (Cost Factor): 12

-

+

High security - suitable for production

Generate Hash

\$2a\$12\$RDMUS/.dNOiz.kJA03WF6.n4F/QRAT7N1lhJjB.i1LSJv/kBEK9hG

Copy Hash

Generated with 12 rounds of hashing

If I generate a new hash with the same exact text, what do you know? The hash is completely different even though we have the same password.

So obviously this is a good thing because the cost for attackers to gain access to your password increases and faulty login attempts will rarely happen and attacks at scale are going to be much less. You want password hashing to be painfully slow for the attackers but not for the users. And salting does just that.

What Happens in a Real Breach?

Database gets dumped

Attacker gets hashes

Offline cracking begins

Weak passwords fall first

So what actually happens in a real breach? How can we tie this all together into a real world example? Basically, this is what happens. Some sort of database gets dumped. It gets leaked and that information is online. The attackers get access to these hashes and they start to begin cracking these hashes offline. The weakest passwords are going to fail first. And once they crack these passwords, they're going to start logging into accounts. But security isn't about preventing breaches. It's about limiting the damage should a breach arise.

Bad vs. Good Password Storage

So let's talk about bad versus good storage. So terrible password storage would be like this.

Password → Encrypt → DB

Password → Salt → Slow Hash → DB

You have a password, plain text password. You encrypt that password and then you put it in your database. Like I said, this encryption is easily reversible if you have the seed phrase. Whereas over here (right part of img), this is a good example because with the password, you make it undergo some sort of salt rounds through a slow hashing algorithm and then you put it in your database and this becomes much much much more secure. There's so many multitudes more secure. I want you to forget that this (left part of img) option over here even exists. Passwords are not meant to be encrypted. Encryption is a great thing and we're going to talk about later in this video when encryption is good and actually necessary. But for passwords, you always hash them slowly and you always make them undergo. Personally, I say at least 12 salt rounds for production, but everyone has their own opinions. I think 12 should be the standard minimum.

Password Verification Flow

So, how does this tie into the verification flow? Obviously, we never want to unencrypt the password. And at no point in the backend do we ever know the original password.

User enters password
↓
Apply same salt + hash
↓
Compare with stored hash

When a user enters their password, we apply the same salt and the same hash and we compare that with the stored hash that we have in the database. If it matches, they get access.

```
export async function hashPassword(password: string): Promise<string> {  
  return await bcrypt.hash(password, 10);  
}
```

```
export async function verifyPassword(  
  password: string,  
  hashedPassword: string  
): Promise<boolean> {  
  return await bcrypt.compare(password, hashedPassword);  
}
```

So, here's an example from a project that I was doing, but it's still relevant. We have a function to hash the password. Take in the password and it just uses brypt to hash it using 10 salt rounds. And then we have a function to verify the password and it takes in the user's password that it enters and then the hashed password and then we simply just compare and it just compares it after we undergo the same hashing.

I feel like you should design these systems with the thought that passwords are already compromised but your job is to make that compromise survivable.

Sessions vs. Tokens

So now hopefully by this time you understand identity and login and we can start to talk about how systems remember identities. So now we get to talk about another very important topic which is sessions versus tokens and JWTs or JSON web tokens.

HTTP and Statelessness

So after a user logs in right the backend needs to remember who that person is. So to go back to part one of this series HTTP at its core is a stateless interaction meaning each request is independent from all of the other requests. There's no memory of previous requests or interactions on the server side. Each request is independent of one another. And so authentication needs to be reattached to every request each time. And so how do we do that? Well, sessions and tokens, these solve the same problem, but they solve them separately, two different ways.

Sessions

Session = server-side stored state

Client holds a session ID

Server looks up session data

A session is a server side stored state. So it's a state that is stored on the server side.

Obviously, a client holds a session ID and the server is going to look up that session data to match it with the client. The flow can be defined like this.

Login -> Session created -> Session ID sent to client (cookie) -> Every request includes the session ID

A user logs in after a successful login. The session is created. That session ID associated with the session is sent to the client usually through some sort of cookie and every request then includes the session ID. and sessions. They're just pointers to server memory or storage.

Tokens and JWTs

Token = self-contained proof of identity

JWT contains claims

Signed to prevent tampering

No server lookup required (by default)

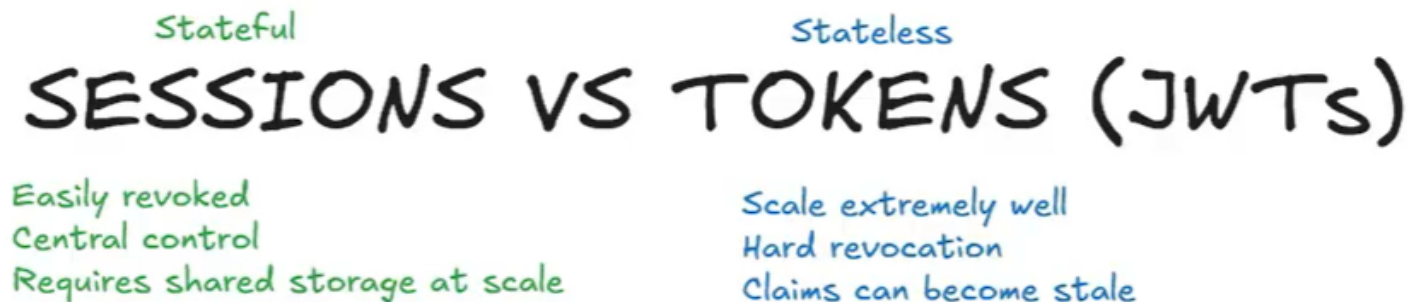
Now, conversely, a token is a self-contained proof of identity. JWT contains claims of who someone is, and it's signed to prevent tampering. And we're going to go super deep into JWTs. So, if these all sound like

buzzwords to you, don't worry. These are all going to be explained. And by default, there's no server lookup required to verify JWTs. And the JWT flow is as follows.

Login -> JWT issued -> Client sends JWT on every request -> Server verifies signature

user logs in, they're issued a JWT. The client then sends a JWT on every request and the server verifies that signature. So JWTs essentially trade server state for client responsibility.

Sessions vs. Tokens: Tradeoffs



So by nature **sessions are stateful** and **tokens are stateless**. This means sessions can be easily revoked and it gives us some sort of central control over them. It also means that they require shared storage at scale. JWTs on the other hand scale extremely well due to their stateless nature. It's hard to revoke these, but that also means that claims can become stale. An important call out here is that stateless does not automatically mean that something is secure by default. But there's no universal winner here. There are different use cases for different instances and circumstances.

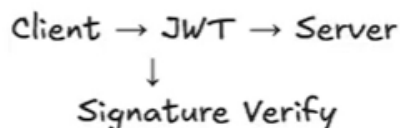
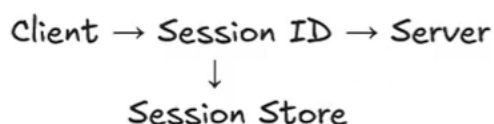
Internal dashboards → sessions

Mobile / APIs → tokens

Hybrid approaches are common

If you have some sort of internal dashboard, you probably want to use sessions. But if you have a mobile app or an API, those most rely on tokens and a hybrid approach is what's most common. And good design is about constraints, not ideology.

Session Flow & Token Flow



So this is a typical session flow. Client is issued a session ID that's sent to some sort of session store and then sent to the server. The ID is meaningless without the server.

On the other hand, for tokens, the client is issued some sort of web token. The signature of that token is verified against some sort of secret, and then the server is sent the token. There's no lookup, and there's no claims inside the token. So, here's just a table explaining different attributes like I put up here, right? Sessions

are revocable. They're server state, and they give you a central control. But web tokens, they're hard to revoke. They're stateless, and they give you a distributed trust.

Sessions	JWTs
Revocable	Hard to revoke
Server state	Stateless
Central control	Distributed trust

You simply just need to choose which to use or if you want to use both based on the constraints of your system and the design of your system.

JSON Web Tokens

So next, let's really dive deep into JWTs and really the internals of how they work. A JWT is not a secure container. It's a signed message.

JWTs don't hide data

They prevent tampering

Anyone can read them

JWTs don't hide data. In fact, they're fully accessible on the front end. They prevent tampering and anyone can read a JWT.

The Three Parts of a JWT

So take an example of a JWT. There's three parts to it. So we have the header, the payload, and some sort of signature or secret. And the header simply just defines what algorithm was used and what the type of the token is. The payload is obviously just data. These are also referred to as claims sometimes. And then the signature. And this signature just acts as a proof that this JWT wasn't modified when we go to decode it.

Encoding Is Not Encryption

So encoding a JWT is not the same as encryption. These are just text friendly bytes. And obviously we can copy paste this JWT into a decoder and it's going to give us back the same data.

So naturally like if we can put this in the decoder that means that it's not encrypted.

Signing vs. Encryption

Signing

Ensures integrity

Detects tampering

Does NOT hide data

Encryption (JWE)

Hides contents

Rarely used

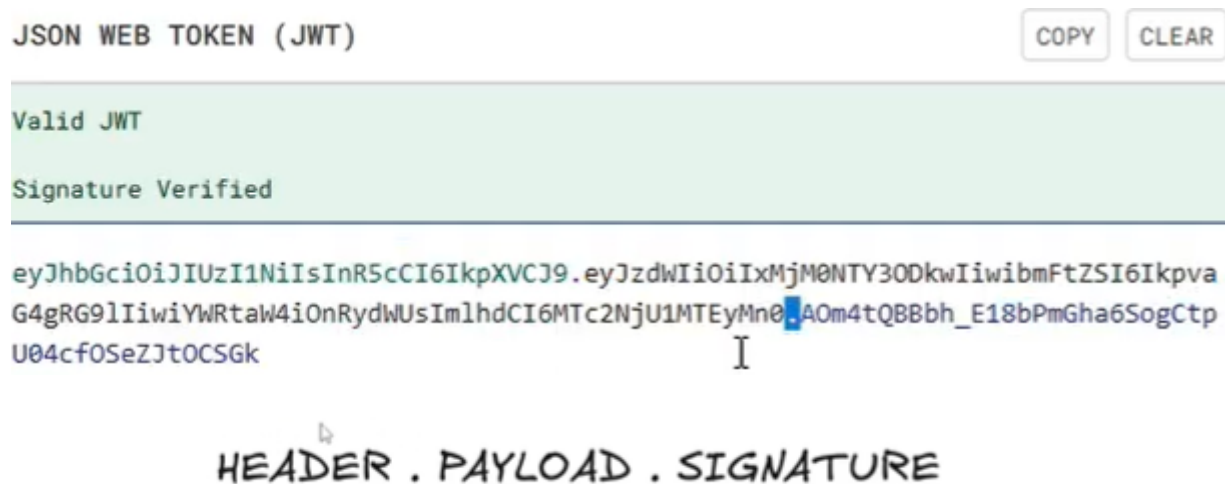
Adds complexity

So what's the difference between signing and encryption and why does it matter? Well signing simply just ensures integrity and it can detect tampering. Okay. When we were using these JWTs at the end of the day to verify that someone is who they say they are and also to make sure that it's valid, of course, right? And so we want to be able to detect any tampering to make sure that it hasn't been modified so that someone doesn't do something malicious. And by nature, since we're signing it, it doesn't hide any data. The data can easily be whereas encryption, it's going to hide the contents and these are rarely used just because it adds complexity and it's unnecessary. Most JWTs you see in the wild are going to be signed and not encrypted.

What Should Never Be in a JWT?

But by nature, obviously, since this is insecure, there are some things that should never be in a JWT. Things like **passwords, API keys, secrets, and things like social security numbers, addresses, any sensitive information**. These should never be inside of a JWT. And the reason is your tokens live in your browsers. Tokens obviously are used to keep track of sessions, right? And these tokens often get logged and the tokens and the traffic associated with them can be intercepted. So you should always assume that your JWT will be read by someone that you didn't intend.

Understanding JWT Structure



So let's look at this. At first, this just looks like three random strings separated by dots. But let's break down exactly what this means. I'm just putting spaces here for simplicity, but there's no spaces here. They're all just separated by these dots as delimiters.

So you have the header which is the first part of the JWT and then you have the payload and then the signature. So here this 1st part before the 1st dot is the header. The middle text is the payload and the last text represents the signature.

Decoding a JWT

JWT → Base64 Decode → Human-Readable JSON

JWT (Client) → Verify Signature → Trusted Claims

So to decode a JWT you just take the token you use some sort of base 64 decoding and then that puts it back into human readable JSON. So the client must undergo some sort of verification to ensure that that signature is legitimate and valid and then we can go on to the trusted claims which is in the payload. We should never automatically trust a JWT unless it's been verified.

Token Lifetimes and Expiration

So tokens have lifetimes. They have a certain span of time where they're valid and after that time they are obviously invalid. And so why does this exist? Why do they expire? And why does it matter?

If a token never expires, then it's essentially like a permanent password to an account. Tokens can get stolen. Uh browsers obviously I've spoken about are untrustworthy and hostile environments. Logs, extensions, cross-site scripting. These are all things that exist and can be abused. Expiration limits the blast radius for these kinds of attacks.

Access Tokens and Refresh Tokens

So we have two kinds of tokens.

Access Tokens

And the first one I'm going to talk about is the access token. An access token deals with authorization. And these should be short-lived. Like it in the span of like minutes and not days. These are used for access to various APIs to do things with certain resources. As we've spoken about, the short lifetime makes them safer.

Refresh Tokens

The other kind of token that we have is a refresh token. Refresh tokens live longer and they're really only used to mint new access tokens.

These also need to be stored more carefully, usually in an environmental variable for the refresh secret.

Access token expires -> client sends a refresh token -> server issues a new access token

When an access token expires, the client sends a refresh token and the server issues a new access token. And these refresh tokens exist to keep access tokens short-lived.

JWT Revocation

Sessions → delete server state

JWTs → already issued tokens live until expiry

And going back to this model of JWT being stateless, how do we deal with revocation? Well, if it's a session, it simply just deletes the server state, right? But JWTs are already issued tokens and these live until their expiry. So this is why lifetimes matter and you want to be very intentional with the lifetimes of your tokens.

*Access Token (short)
Refresh Token (long)*

So, here's a token pair, right? Access tokens are shortlived. Refresh tokens are longlived. Reason being, like you don't you don't want to have to relog into your account every 30 minutes, right? Let's say you're on Twitter or YouTube or something. You shouldn't have to relog-in every 30 minutes. And so, that's why this is the flow up here.

Logout and JWTs

But, this raises an issue. So, what happens when a web token is issued and then the user logs out? Does the JWT just magically disappear?

~~JWT magically disappears?~~

The logout is fundamentally not a protocol feature. And so this is why we need token lifetimes and we need to be intentional with the lifetimes that we set.

Authorization: The Real Gatekeeper

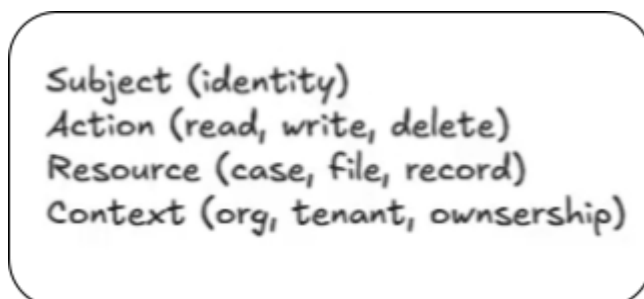
But that's really like the simple part of it. So the next major thing that we need to talk about that I'm going to go pretty in depth on is authorization which is the real gatekeeper. You know authentication is rather simple but authorization. This is the true gatekeeper when it comes to what sort of requests succeed and what don't.

So, as I've talked about, authentication tells you who is making the request, but authorization decides whether that request should actually succeed. And most security fails after authentication happens.

Authorization Is a Runtime Decision

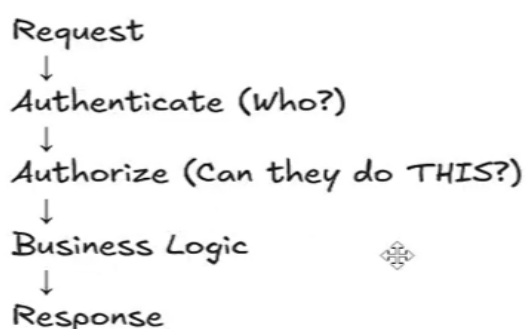
You might think that authorization looks something like this. If the user is an admin, then we allow them to do something, right? Authorization is just a decision that's made at runtime using some sort of context.

Authorization Decisions



So, we have a broad amount of authorization decisions, right? And if you're missing any of these inputs, then your Auth logic is incomplete. Every time a login happens, tokens are reused, requests keep coming, you're going to do some sort of authorization even if they have a valid JWT, even if a session exists. And even if a user logged in two seconds ago, you still must authorize. As I've stated, a valid token only proves identity. It does not prove permission to do something.

Authorization Request Lifecycle



Here's the request life cycle in an authorization sense. The user sends a request, we authenticate them, figure out who they are, then we authorize them. Can they actually do this function right here? This is the important part. Once we actually figure out, okay, they have the permissions to do this, then our business logic will run and we'll get a response out of it. If the 3rd step is missing, everything after this becomes a liability.

Global and Resource-Level Authorization

Going back to what I said earlier about authorization being resource specific. We have two different types of authorization. We have a global authorization and a resource level authorization.

GET /cases/123

So suppose we have this path. Okay. We call a get method on the path of cases of the specific case number 1 2 3. We need to ask if this user is authorized on what and on which case. Okay.

Allowed to read a case

!= being allowed to read THIS case

Just because a user is allowed to read a case, it can be allowed to access cases, that doesn't necessarily mean that they are allowed to read this specific case. And common failures arise when you only check someone's role or you only check their org membership and you don't check ownership of a specific resource.

You can't just assume that because someone has access to this more global resource that they have access to this sub resource within whatever they have access to and authorization without this resource awareness is a major security downfall.

Good vs. Bad Authorization

if (user.role === "lawyer") allow

So let me show you two examples. One of some good authorization and some bad authorization. If we just check to see if a user has a specific role and just automatically allow something,

Authorization Decisions

Subject (identity)
Action (read, write, delete)
Resource (case, file, record)
Context (org, tenant, ownership)

~~*if (user.role === "lawyer") allow*~~

this doesn't cover the rest of these authorization decisions. We don't check to see if they can perform specific actions. We don't check resource specific permissions. We're just checking to see if they have a role. This doesn't envelop the full scope of things.

Does user belong to org?
AND
Does user have permission to read cases?
AND
Does user have access to THIS case?

A better example of this is does the user belong to this organization and does this user have permission to read cases and does the user have access to this specific case.

We're checking all of the edge cases and ensuring that they have every single required permission.

Authentication Does Not Equal Authorization

So to branch off of that, I'm going to give you a real example of where authentication does not equal authorization. Suppose this scenario arises, right? A user logs in successfully. Their JWT is then validated and it is valid and the token says that they have a role as a user. They then send a get request on the user's path of user 456.

User logs in successfully
JWT is valid
Token says role: "user"

GET /users/456

Do we know which user this is? Right? Just because they're authenticated, that doesn't mean that they're authorized. The server still must ask should this user be able to see that user and while authentication is obviously necessary it's just not sufficient when it comes to authorization and allowing resource control. So don't just have the mindset that because someone is authenticated they should be allowed to do something. Instead only if both of these are true then we can allow them to do something.

Authorization Models

So why do teams struggle with authorization? Well, like there's a ton of different reasons, but like everything can be boiled down to scaling, right? Like if you have different rules that are growing over time or new roles get added, you have exceptions that pile up or even the worst of them all, the worst offender being business logic leaking into Auth logic. These are all going to be pitfalls for your authorization. A common theme in this course I want to be is separation of concerns. The more you can separate things on the back end and separate the concerns of files and different layers, the happier you're going to be and the easier it's going to be to first prevent bugs and secondly debug them.

But naturally, authorization complexity grows with your business model and this leads into a necessity for some sort of role-based access control or action-based access control or some sort of ownership model.

Why Authorization Models Exist

So let's talk about it, right? Authorization models, these exist because ad hoc permission checks don't scale. Every serious system is going to end up with an authorization model of some sort. So what are these models, right? Let's break these down.

Role-Based Access Control (RBAC)

So our backer role based access control answers a question for us. **What can someone with this role do?** We have specific roles for users, whether they're an admin, a viewer, or some sort of editor. And we have permissions that are attached directly to the roles. And then obviously users are assigned these roles. For example, like with the SAS that I'm making for the law firms, right? We might have a lawyer role, a paralegal role, and an admin role all with separate permissions to do things. So these work well when like you have a small organization. The rules are fairly simple and there are going to be few exceptions.

This is a use case when this works well.

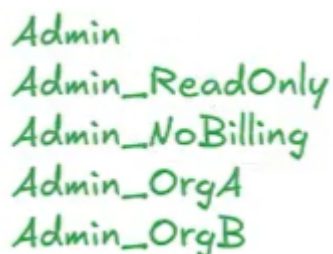
Basic RBAC Model

So this is very basic, right? You have a user and we check the role and then we check the permissions of that specific role. There might be some permissions in this role like **read_case**, **edit_case** or **delete_case**. RBAC simply centralizes permissions which is why it's so attractive early on.

Where RBAC Starts to Break

But this slowly starts to crack. They could be an admin but might not be that admin. You know what I mean? What if someone they might be like a viewer, but what if what if the viewer owns a specific record? They should have edit control, but what if that what if that permission is not a part of the role they're a part of? When you have role explosion and you need to have specific roles for like an admin or a super admin, org admin, case admin, it gets messy because permissions get duplicated and no one knows what the roles actually mean anymore.

And so fundamentally, RBAC breaks when permissions depend on context.

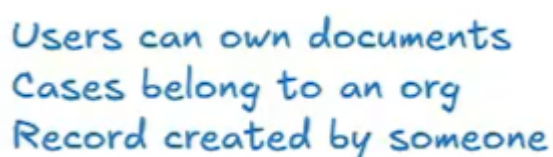


Admin
Admin_ReadOnly
Admin_NoBilling
Admin_OrgA
Admin_OrgB

I mean, this is just so verbose right here. like there's no need for this. But when you use the RBAC model and the complexity and the scale of your app grows, like this is what you're going to end up having to do unfortunately. And this is just unmanageable.

Ownership Model

So let's talk about an ownership model next. Ownership finally answers the question of is this resource is associated with a specific user or entity.



Users can own documents
Cases belong to an org
Record created by someone

So like users can own documents, cases belong to an organization and like records can be created by someone and ownership isn't necessarily a role. It's just as the name implies, it's just an ownership of specific resources. And so you can be a non-admin and still edit your own resource.

Non-admin -> edit your own resource

User -> Resource

resource.ownerId === user.id

And so it's a very simple flow. We just check the user and see if they have access to that resource.

And that can look something like this in some pseudo code. And most authorization decisions quietly rely on this ownership model.

Attribute-Based Access Control

The last one I want to talk about is attribute or action-based access control. And ABAC uses attributes and policies instead of fixed roles.

User attributes (role, department)
Resource attributes (status, owner)
Environment attributes (time, location)

So attributes can be user attributes like role or department. They can be resource attributes like status or owner and they can be environment attributes like time or location. And essentially what this is saying is allow access if a user belongs to an organization and resource status is active and action is read. So the model kind

of follows like this. **Attributes -> Policy Engine -> Decision** It has a list of attributes and then goes through some sort of policy engine and then makes a final decision based on that. Things like user, resource, context all fall into this. ABAC trades the simplicity for flexibility.

Layering Authorization Models

So how do the real systems work and actually use these? Almost no system uses pure role-based access control. Most use **RBAC for coarse access**, **ownership for specificity** and **ABAC for edge cases**. And the key point I want you guys to remember is these authorization models. They don't replace each other but rather they layer on top of one another to create a complete whole system.

Object-Level Authorization

So now let's look at where this authorization fails in most cases. So object level authorization is where most breaches happen. So naturally, most bugs are not about roles, but they're about specific resources. This is where systems leak data. Object level authorization simply asks whether a subject can access this specific instance of a resource. So like this specific case, this specific document, this user profile. Instead of asking like can a user read cases, we ask the question like can this user read this case.

Objects vs. Types

Object

Type

Case #123

Case

So we kind of need to cover objects and types. So a case is just a type but the specific case one two three is an object and attackers don't target types they target ids which fall under objects.

IDOR

So there's a classic bug with this IDOR or insecure direct object reference. And basically you can think about it like this scenario. A user is logged in and then they send this request.

User is logged in
Requests:

GET /cases/123

GET /cases/124

They send a get request on this case but then they might try to get the next case. If the server only checks authentication you get a data leak. And if the server only checks the roles you still get a data leak. We have to check access on not just the cases as I've stated, but we need to check the access on each specific ID, each instance of whatever object they're trying to access.

Authenticated User

↓
GET /cases/123 → Allowed

↓
GET /cases/124 → Should fail ❌

If changing this ID changes the data you're trying to access, then you need an authorization check. And assuming that this user doesn't have access to this resource, the authenticated user should be able to get 123, but it should fail when they try to get 124. That's pretty much it. I don't want to be redundant and go over things I've talked about. So, that pretty much just branches off of that.

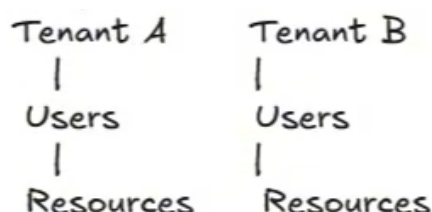
Multi-Tenancy

But the next thing I want to talk about is multi-tenancy in applications. So, multi-tenant authorization is an absolute minefield, especially in SAS. But to be honest, it really just is object level authorization at scale. And one missed check with this can literally be a company ending breach.

What Is Multi-Tenancy?

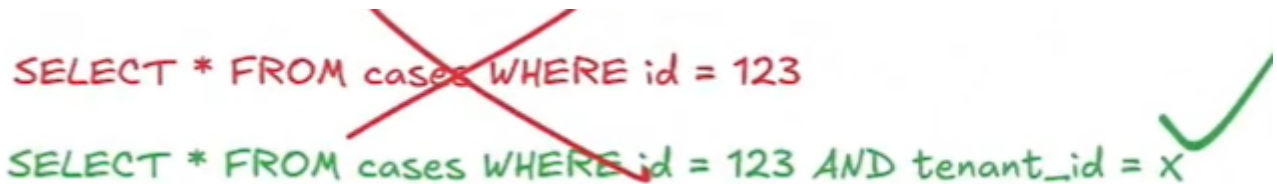
To put it in plain English, multi-tenancy simply means that **multiple organizations share the same application and infra, but they must behave as if they are isolated**. So these tenants are all sharing the same code, but of course they all have data that needs to be completely separate from one another. And for example, let's say like my example, if you're developing a back-end software for law firms, you don't want law firm A to be able to access law firm B's data and vice versa.

Tenant Context



So how we keep track of this is we have the tenant model right we have tenant A and tenant B law firm A and law firm B they all have some users and some resources the separation here is what tenant these are context always comes first here and every request must carry the context of which tenant is trying to do something tenant ids must be delivered on the server side and you never want to trust client supplied tenant IDs because any malicious attacker could forge a tenant ID and that's really bad especially if you're in a heavily compliant niche where a breach is genuinely like a crime. So we have a request:- user ID tenant ID role like this tenant ID this is perhaps the most important part of this in a multi-tenant architecture.

Tenant-Scoped Queries



~~SELECT * FROM cases WHERE id = 123~~

SELECT * FROM cases WHERE id = 123 AND tenant_id = X ✓

So suppose we have this multi-tenant bug like two of these queries right this one only has one more parameter it only has one more clause but this is genuinely the difference between a genuine safe query and total data exposure.

Authorization Implementation Patterns

So, now we'll get into some implementation patterns for authorization and how to not screw this up. So, oftentimes it's an architecture or an implementation problem that causes bugs in a system, not with business logic, but with how you implemented something. Where you decide to put this authorization matters just as much as how you write it.

Inline Authorization

```
if (user.role !== "admin") {  
  throw new Forbidden()  
}
```

So, let me show you guys first the worst pattern in all of existence. This is called inline authorization. And this genuinely makes me sick to my stomach. Like this right here is genuinely impossible to audit. And this just gives inconsistent behavior. You're pretty much just copy pasting security. And this fails over time because new rules can get thrown into one place. And tests can miss edge cases while old paths get forgotten.

```
Controller  
  if (...)  
Service  
  if (...)  
Repo  
  if (...)
```

This is just this is a mess. To be real, if in your controller, your service, and your repo, you need if statements to check roles or control, this is just so messy and too verbose.

Centralized Authorization Layer

So that's where the centralized authorization layer comes in. All authorization should go through one path. Like I've stated so many times, you need a clean separation of concerns, and this is no different. Your authorization should be in one central location. Perhaps you have some sort of function that authorizes someone based on who they are, what action they want to take, and what resource they're trying to access.

`authorize(user, action, resource)` This leaves things up to one model, one place to test things, and one place to audit should something go wrong. Authorization should be boring and predictable like this.

Request → AuthZ Gate → Business Logic

Authorization Gate

The flow should look something like this. When you have a request, it should go through some sort of authorization gate before the business logic runs. And you should really be using policy objects instead of conditionals because like over here, if you have all these if statements, it just gets messy.

Policy Objects

```
CasePolicy.canView(user, case)
CasePolicy.canEdit(user, case)
```

```
CasePolicy
- canView
- canEdit
- canDelete
```

If you have named policies that are explicit, these naming rules force you to understand what this is doing and centralize the authorization logic.

So this case policy object might have some methods like can view, can edit, can delete. And this keeps things nice and neat.

```
{
  "allowed": false,
  "reason": "not_case_owner"
}
```

And your authorizations, they should also return structured reasons, not just we allow or disallow this. And the reason for this is because it leads to a better user experience, better logs when it comes to debugging. And it just makes your life so much easier when you come into an issue at 2 in the morning and you have no idea what you're looking for because you didn't include this field within your return.

Layered Checks

Controller → Service → Query (tenant scoped)

You can also implement some sort of layered check. So if you have a controller or a handler that sends a request to some sort of service and that service queries something, these should all be tenant scoped.

Like I stated, all should be tenant scoped at the base of it, right? And each of these instances could have a check. So the main point that I want you guys to take away is that **authorization is a system, not just a set of if I if statements**.

Testing Authorization

So far I've talked about a ton of bugs and issues that can arise with authorization. But how do we actually test and ensure that we have a secure system? Well, there's three main categories that you should be testing.

Role-Based Denials	Object-Level Attacks (IDOR)	Tenant Isolation
Viewer tries to edit Member tries to do an admin action	Access objects you don't own Same role, different resources	Same user ID Different tenant Same endpoint

1. Role-Based Denials

We have role-based denials. Examples of this can be a viewer tries to edit something or a member tries to do an admin action and hence the name denials. Every permission should return a negative test.

2. Object-Level Attacks

The next bucket is object level attacks. Trying to access an object that you don't own or users with the same role but with different resources. Changing IDs should not change authorization.

3. Tenant Isolation

And the last category is tenant isolation. We'll check to see if two users have the same ID, but they're a part of different organizations or tenants accessing the same resources, with across different tenants or the same endpoint across different tenants.

Tenant leaks, as I've said, are catastrophic, especially if you're developing an infrastructure for a SAS.

Authorization Test Matrix

So the real test matrix is something like this. This is your real test service. You check **the user, the role, the tenant, and the object**. And I want you guys to adopt a property-based thinking model. Instead of testing specific cases, you want to test invariance. Examples of this can be things like a user can never have access to another tenants's data or a non-owner can never modify a resource. Instead of testing routes, you want to test rules.

Capstone: Authentication and Authorization

So, let's tie it all together with this capstone. Most broken systems, their security fails simply because authentication and authorization get mixed together. As I stated at the very beginning of this video, these two things solve different problems and they should be treated that way.

Authentication

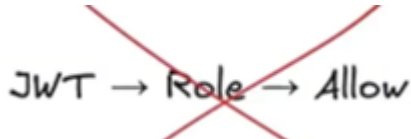
Authentication answers one question. **Who are you?** This ties into identity proof, credentials, tokens, and sessions. Auth is about identity not permission. The very simple flow of this is you have a client. They prove

their identity and they are supplied with a user ID and they're matched with that user ID. This is the only thing Auth deals with.

Authorization

Authorization on the other hand answers what you are allowed to do? This is highly contextual, highly dynamic and resource specific. Authorization is about decisions not identity. And the authorization flows as follows. you have a user, a tenant that's trying to do an action with a specific object, you either allow or deny that process.

The Clean Boundary Pattern



This merge of things is terrible because you don't have a separation of concerns here. You're just checking to see is this JWT valid? Do they have a role? Allow it without checking anything else. The clean boundary pattern follows this exact flow. Okay? You authenticate once, you extract their identity, and you authorize every action. You want to authenticate rarely, like I said, to prevent having people log in every 20 minutes, but you want to authorize constantly. The clean flow looks something like this.



Someone sends a request, you authenticate them, you then authorize what they're trying to do, and finally, you execute that action if they're authorized.

Final Mental Model

Final mental model, guys.

Authentication tells you who someone is.
Authorization decides what happens next.

Authentication tells you who someone is. Authorization decides what happens next.

If you get this right, everything else becomes easier.

If you get this wrong, then literally nothing else matters.